

Projekat: "SIFT, algoritam za izvlačenje feature-a"

Dokumentacija za predmet: Projektovanje elektronskih
uredjaja na sistemskom nivou

1. Uvod

Pronalaženje slika koje se podudaraju je jedan od čestih problema u computer vision-u. Pomoću algoritama koji traže sličnosti u slikama, mogu se rešiti razni problemi kao što su: prepoznavanje različitih scenarija, prepoznavanje 3D struktura unutar više slika, praćenje pokreta, i tome slično.

“SIFT” (Scale-Invariant Feature Transformation) je jedan od algoritama koji se obilato koriste u computer vision-u u te svrhe. On se zasniva na pronalaženju takozvanih feature-a odnosno karakteristika slike. Ovi feature-i se ne menjaju u zavisnosti rotacije slike, njene razmere, osvetljenja, šuma ili sličnih pojava. Iz ovog razloga, dobro izvučeni feature-i mogu biti ključni za rešavanje gore navedenih problema.

Da bi pronašao ove karakteristike, algoritam prolazi kroz 4 ključna koraka:

1. Detekcija ekstrema u skali i prostoru. Ovo se radi tako što se prolazi kroz sve skale odnosno slike različitih razmera i traže se ekstremi. Ovo je najefikasnije implementirano DoG (Difference-of-Gaussian) algoritmom.
2. Lokalizacija ključnih tačaka (feature-a). Nakon što su ključne tačke pronađene tako što su poredjene sa njihovim susednim pikselima, sledeći korak je da se one detaljno prilagode okolnim podacima zbog lokacije, skale i zakrivljenosti. Ovo je važno da bi se neke tačke koje su slabog kontrasta, osetljive na promene ili se nalaze pored ivice odstranile.
3. Dodela orijentacije. Svako ključnoj tački je dodeljena jedna ili više orijentacija pomoću kojih se predstavlja. Sve naredne operacije se vrše nad informacijama iz ovih dodeljenih orijentacija, skale i lokacije ključne tačke. Iz ovog razloga su ključne tačke nezavisne od transformacija kao što su približavanje, udaljavanje ili rotacija slike.
4. Nakon ovih operacija, svaka ključna tačka je nedvosmisleno određena u 2D prostoru. Sledeći korak je izračunati *deskriptor* za region u oblasti ključne tačke koji je veoma karakterističan i nepromenljiv u drugim transformacijama kao što je promena osvetljenja ili promena 3D tačke gledišta.

Ovaj pristup se naziva Scale-Invariant Feature Transformation (SIFT) zato što transformiše podatke iz slike (vrednost piksela) u kordinate povezane sa lokalnim ključnim tačkama koje su nezavisne u odnosu na skalu (gledište).

Ovaj algoritam je veoma efikasan jer daje veliki broj gustih ključnih tačaka. Broj tačaka zavisi od raznih parametara koji se koriste u izračunavanju ali sa standardnim parametrima koji se najčešće koriste i pogodni su za slike različitih tipova, za tipičnu sliku veličine 500x500 piksela, dobija se čak i do 2000 ključnih tačaka (Ovaj broj svakako zavisi od sadržaja slike).

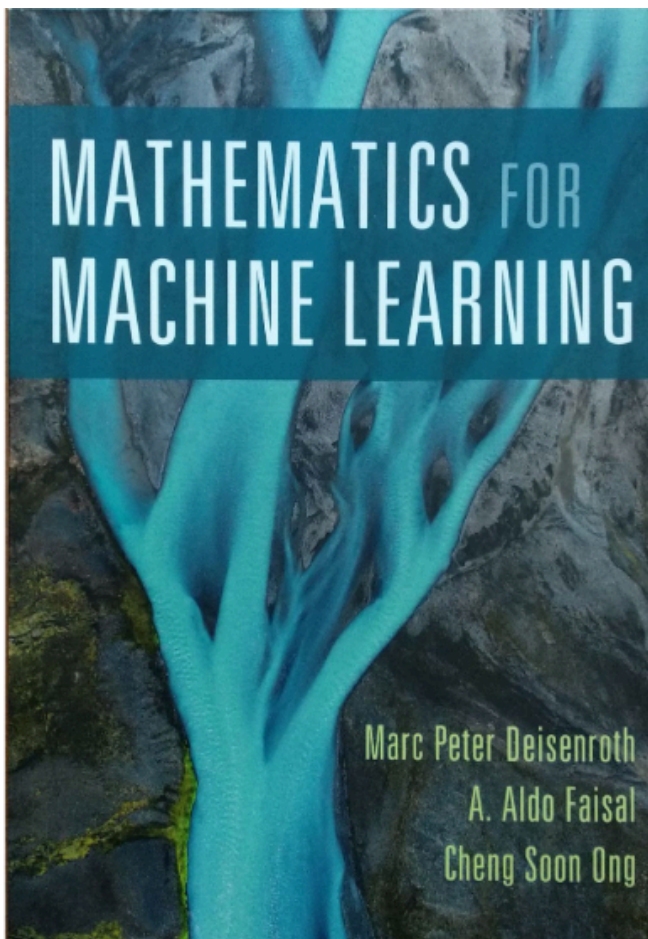
Za pronalaženje slika koje se podudaraju, što je najčešći primer upotrebe ovog algoritma, feature-i se prvo izvlače iz set-a referentnih slika i smeštaju se u bazu podataka. Kada je potrebno proveriti da li neka slika sadrži tražene objekte, ovo se radi tako što se feature-i na novoj slici redom upoređuju sa onima iz baze podataka.

2. Specifikacija

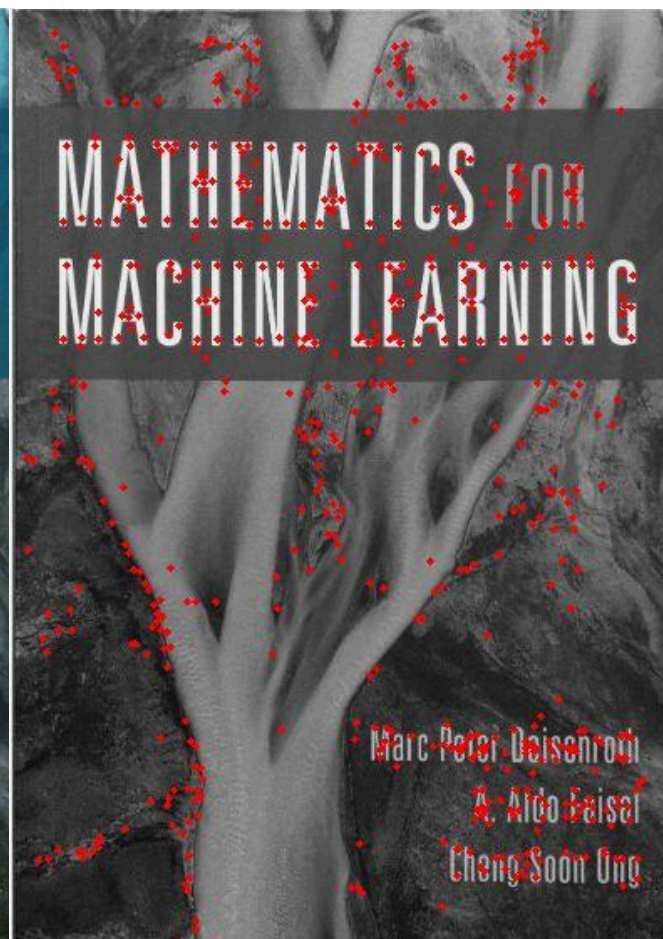
Izvršna specifikacija je kucana u programskom jeziku C++ i implementiran je ceo postupak koji je naveden u uvodu. Ulazni argument pri pokretanju izvršne specifikacije je putanja na sliku nad kojom želite da izvršite SIFT algoritam a izlaz se čuva kao result.jpg i na njemu su prikazane sve ključne tačke koje su pronađene primenom algoritma kao crvene tačkice na originalnoj slici (Prebačenoj u *grayscale*).

Pre nego što se kompajlira specifikacija, moguće je menjati parametre specifične za SIFT algoritam ali su oni već namešteni tako da algoritam radi što efikasnije (Neki od ovih parametara su informacije kao što je broj oktava, broj slika po oktavi, količina *blura* za svaku narednu sliku...).

Primer rada:



Slika 1. Ulazna slika knjige



Slika 2. Rezultat obrade

3. Rezultati profajliranja

SIFT se sastoji iz veoma velikog broja koraka i različitih računskih operacija od kojih neke rade nad originalnom slikom, neke na nizu slika koje su generisane od prethodnih koraka a neke na već generisanim ključnim tačkama. Svi ovi koraci prave različite transformacije i računanje kako bi se došlo do konačnog rešenja.

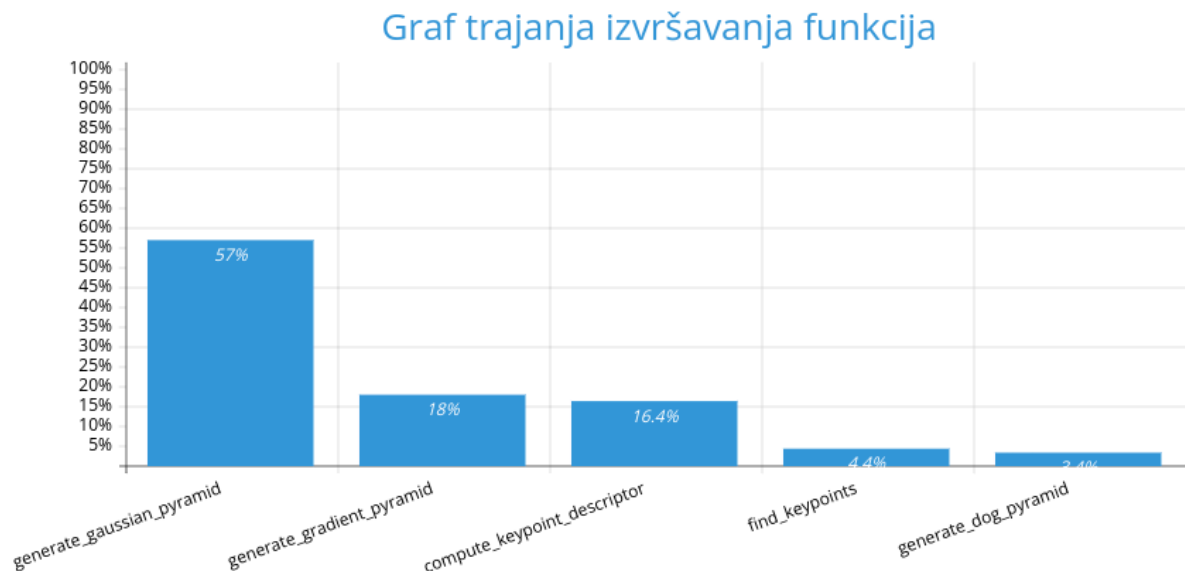
Iz ovog razloga, najjednostavniji način za proveru performansi različitih delova koda je profajliranje.

Profajliranje je radjeno pomoću GNU profajlera (gprof). I rezultati profajliranja mogu da variraju od veličine slike i parametara koji se koriste za algoritam. Jedan od parametara od kog zavise rezultati profajliranja je broj oktava koji se koristi a on se odredjuje u zavisnosti od veličine slike za koju je potrebno naći ključne tačke. Ako je slika veća, na primer 4K formata (3840x2160 piksela), potrebno je veci broj oktava, na primer 8, jer su slike u svakoj sledećoj oktavi dva puta manje. Ako radimo sa manjim slikama, na primer formata 256x256, broj oktava koji je dovoljan može biti i 4.

Profajliranje je vršeno sa brojem oktava postavljenim na 8 i nad većim brojem slika čija veličina je bila ispod dimenzija 256x256 piksela.

Rezultat profajliranja je pokazao da u proseku funkcija **generate_gaussian_pyramid** oduzima 57% vremena, funkcija **generate_gradient_pyramid** oduzima 18% vremena, funkcija **compute_keypoint_descriptor** oduzima 16.4% vremena, funkcija **find_keypoints** oduzima 4.4% vremena, funkcija **generate_dog_pyramid** oduzima 3.4% vremena.

Ova raspodela je prikazana na tabeli ispod.



Nakon ove analize došli smo do zaključka da ovakav algoritam nije pogodan za hardversko ubrzanje ali u akademske svrhe i svrhe ovog projekta odlučeno je da će se ubrzavati najsporiji deo algoritma.

Analizom funkcije **generate_gaussian_pyramid**, uvideli smo da od 57% vremena, 55% vremena oduzima funkcija **gaussian_blur**, i ona će biti hardverski ubrzana.

4. Bitska analiza

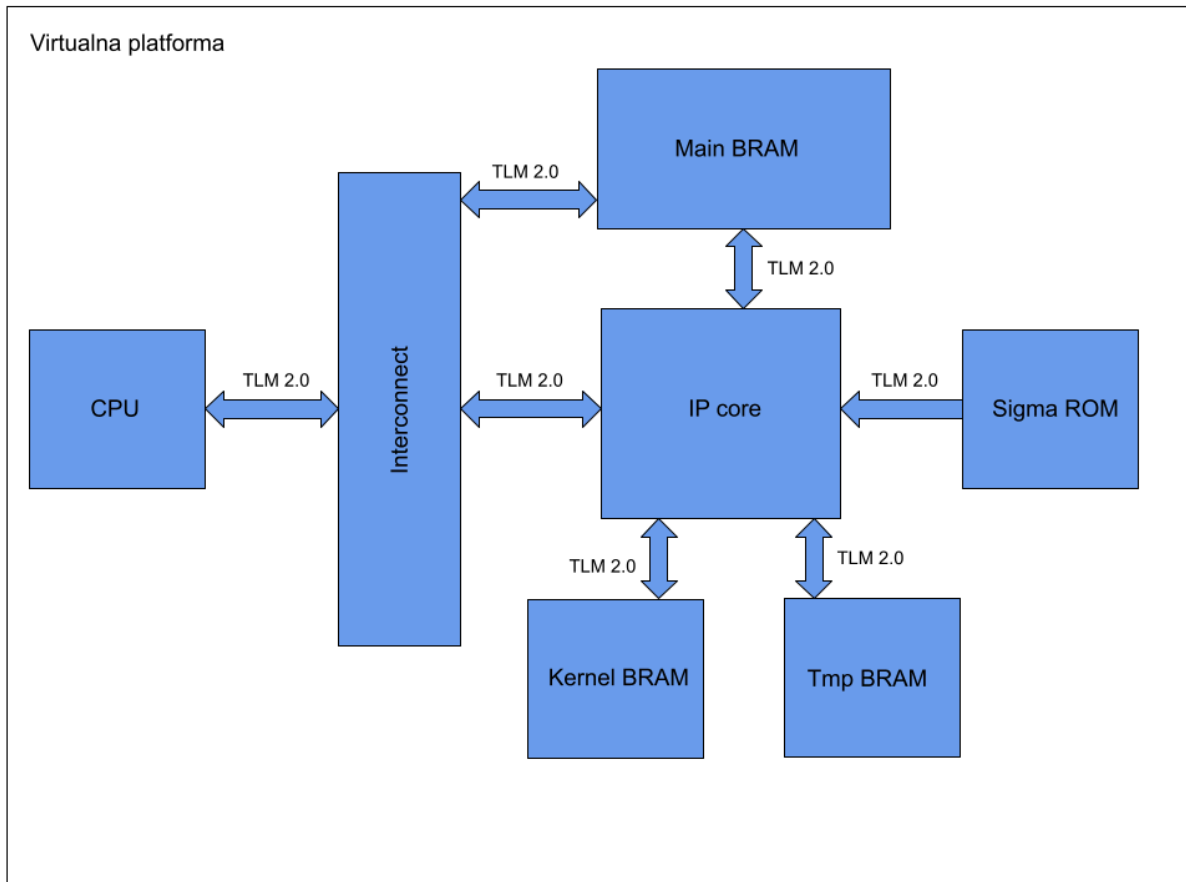
Nakon analize programa u C++, zaključeno je da se koristi nekoliko promenljivih koje su tipa float. Pošto je ovo tip podataka koji reprezentuje brojeve sa pokretnim zarezom, za ove promenljive su definisani novi tipovi podataka koji koriste reprezentaciju sa fiksnim zarezom i daju zadovoljavajuće rezultate pri radu.

1. Prvi i najvažniji od ovih tipova je **typedef sc_dt::sc_fixed_fast<16, 2> data_t;** Ovaj tip se koristi za predstavljanje vrednosti svakog piksela. U našem algoritmu pikseli su predstavljeni kao realne vrednosti gde se vrednosti piksela prvo dele a onda se prebacuju u crno bele pošto algoritam nastavlja da radi sa crno belim slikama.
2. Drugi tip je **typedef sc_dt::sc_ufixed_fast<16, 1> sum_t;** i ovaj tip se koristi za smeštanje vrednosti piksela prilikom konvolucije sa kernelom u funkciji `gaussian_blur`.
3. Treći tip je **typedef sc_dt::sc_ufixed_fast<32, 9> sigma_t;**. Ovaj tip se koristi da se u njega skladište vrednosti sigme sa kojom se vrši blurovanje slike.

Postoje još neke promenljive koje se koriste u harveru i koje imaju svoje tipove ali ti tipovi su dosta jednostavniji i nije bila potrebna analiza da bi se došlo do zaključka koja je njihova veličina.

Zbog ograničenja BRAM-a na zybo ploči i velike memorijske zahtevnosti algoritma, odlučili smo se da delimo slike na više delova i da ne obradjujemo prevelike slike. Slike smo ograničili na 256x256 piksela ali postoji mogućnost nameštanja parametara na koliko delova se slika deli, ukoliko je potrebno obradjivati veće slike.

5. Virtuelna platforma



Slika 3. Blok šema virtuelne platforme

Na slici 3 je prikazana blok šema virtuelne platforme sistema koji izvršava funkcionalnost, odnosno izvršava ceo SIFT algoritam. Virtuelna platforma se sastoji iz nekoliko komponenti a to su:

1. CPU modul koji predstavlja softverski deo. On izvršava kod pre i nakon dela koji se ubrzava u harveru.
2. Interconnect modul koji vrši memorijsko mapiranje registara i povezuje CPU sa ostatkom sistema.
3. IP Core je hardverska komponenta koja će biti napravljena na FPGA delu i koja izvršava kritični deo programa odnosno gaussian_blur funkciju.
4. Main BRAM je deo memorije na zybo-u koji skadišti sliku koja se obradjuje.
5. Kernel BRAM je deo memorije na zybo-u koji skadišti kernel sa kojim se vrši konvolucija.
6. Tmp BRAM je deo memorije na zybo-u koji skadišti privremenu sliku izmedju dve konvolucije.
7. Sigma ROM je look-up tabela koja skladišti potrebne vrednosti sigme sa kojima se vrši blurovanje.

Sistem radi na taj način da CPU odnosno deo programa koji se izvršava u softveru učitava sliku i izvršava njenu pripremu za Gausovo blurovanje. Kada dodje do dela gde je potrebno da se izvrši Gausovo blurovanje slike, CPU namešta registre IP Core-a koji se koriste za prihvatanje parametara koji su potrebni za rad IP Core-a. CPU takodje u Main BRAM upiše vrednosti piksela slike kojoj je potrebna obrada.

Nakon toga hardverska komponenta počinje sa radom. Hardverska komponenta prvo učitava vrednost sigme iz ROM-a i u odnosu na tu sigmu računa vrednosti kernela koje smešta u BRAM namenjen za kernel matricu. Nakon toga radi konvoluciju po y-osi i x-osi uspomoć kernela koji je već smešten u BRAM i time bluruje sliku. Tmp BRAM služi za skladištenje slike nakon što izvrši konvoluciju po y osi. Kada je izvršena konvolucija po y-osi, vrednosti iz Main BRAM više nisu potrebne pa se vrednosti konvolucije mogu upisivati u Main BRAM. Nakon što su obe konvolucije završene, IP core javlja da je završio a CPU izvlači vrednosti iz Main BRAM-a uspomoć interconnect modula.

Ovaj postupak se ponavlja više puta da bi se generisala piramida slika koja se kasnije koristi u nastavku SIFT algoritma (Postupak se ponavlja $\text{broj_oktava} * \text{broj_slika_po_oktavi} + 1$ puta. Ukoliko je slika prevelika i mora da se podeli, ovaj broj se još pomnoži sa brojem na koliko delova se slika deli).

CPU i IP Core komuniciraju uspomoć ready, start i reset bita.

6. IP Core modul

Registri

Naziv registra	Adresni ofset	Širina registra	Opis
ADDR_IMG_WIDTH	0x00	16 bita	Širina fotografije
ADDR_IMG_HEIGHT	0x01	16 bita	Visina fotografije
ADDR_IMG_OFFSET_UP	0x02	16 bita	Broj dodatnih redova koji su prosledjeni jezgru a nalaze se po y osi iznad dela slike koji se obradjuje
ADDR_IMG_OFFSET_DOWN	0x03	16 bita	Broj dodatnih redova koji su prosledjeni jezgru a nalaze se po y osi ispod dela slike koji se obradjuje
ADDR_NUM_IMG_OCT	0x04	16 bita	Redni broj slike u oktavi pomoću kojeg se bira koja se sigma koristi za blurovanje
ADDR_RESET	0x05	1 bit	Bit pomoću kog se resetuju jezgro
ADDR_START	0x06	1 bit	Bit kojim se označava da jezgro treba da krene sa radom
ADDR_READY	0x07	1 bit	Bit koji označava da li je jezgro trenutno zauzeto

Tabela 1. Tabela registara IP jezgra

U tabeli 1 su prikazani svi registri IP jezgra pomoću kojih se jezgro konfiguriše i pomoću kojih se komunicira sa jezgrom. Dati su nazivi, adrese, širina i kratak opis svakog od registra.


```

Image gaussian_blur(const Image& img, sigma_t sigma, uint16_t offset_up , uint16_t offset_down, uint16_t j)
{
    int size = std::ceil(6 * sigma);

    if (size % 2 == 0)
        size++;

    int center = size / 2;

    Image kernel(size, 1, 1);
    sigma_t sum = 0;
    for (int k = -size/2; k <= size/2; k++) {
        num_t val = std::exp(-(k*k) / (2*sigma*sigma));
        kernel.set_pixel(center+k, 0, 0, val);
        sum += val;
    }

    for (int k = 0; k < size; k++){
        kernel.data[k] /= sum;
    }

    Image tmp(img.width, img.height-(offset_up + offset_down), 1);
    Image filtered(img.width, img.height-(offset_up + offset_down), 1);

    // convolve vertical
    for (uint16_t x = 0; x < img.width; x++) {
        for (uint16_t y = offset_up; y < img.height - offset_down; y++) {
            num_t sum = 0;
            for (int k = 0; k < size; k++) {
                int dy = -center + k;
                sum += img.get_pixel(x, y+dy, 0) * kernel.data[k];
            }

            tmp.set_pixel(x, y-offset_up, 0, sum);
        }
    }

    // convolve horizontal
    for (uint16_t x = 0; x < img.width; x++) {
        for (uint16_t y = 0; y < tmp.height; y++) {
            num_t sum = 0;
            for (uint8_t k = 0; k < size; k++) {
                int8_t dx = -center + k;
                sum += tmp.get_pixel(x+dx, y, 0) * kernel.data[k];
            }

            filtered.set_pixel(x, y, 0, sum);
        }
    }

    return filtered;
}

```

Slika 4. Kod funkcije gaussian_blur

Na slici 4 je prikazan kod funkcije gaussian_blur iz bitske analize. Kod se može podeliti na 3 glavne celine a to su:

- Izračunavanje vrednosti kernela u zavisnosti od sigme (Ovaj deo je moguće izbaciti jer se radi sa *const* vrednostima i može se zameniti sa čitanjem ROM memorije).
- Vertikalna konvolucija slike uspomoć izračunatog kernela.
- Horizontalna konvolucija slike uspomoć slike koja je nastala nakon vertikalne konvolucije i kernela.

7. Procena performansi

Za potrebe procene performansi bilo je potrebno izračunati vremena u izvršavanju hardverskog dela i pristupa hardvera BRAM memoriji ali i vreme pristupa softvera BRAM-u da bi prvo napunio BRAM podacima potrebnim za obradu od strane hardvera a kasnije da bi te podatke izvukao iz BRAM-a.

Da bi ovo moglo da se proceni bilo je potrebno modelovati najšloženiji deo sistema pomoću VHDL-a i napraviti sintezu. Nakon siteze dobijeno je da najduža putanja traje 9ns što je ekvivalentno radu na frekvenciji od 111Mhz.

Kada je dobijena ova veličina koja je u kodu nazvana *delay*, dodate su 3 promenljive. Jedna koja akumulira vreme pristupa softvera BRAM-u a druga koja akumulira vreme izvršavanja hardvera i pristup BRAM-u od strane hardvera.

Vreme izvršavanja naše funkcije u hardveru dobijamo tako što *offset_hard* promenljivu uvećavamo na svim mestima gde smo pretpostavili da će preći jedan ciklus u radu i na svakom mestu pristupa BRAM-u po jedan ciklus odnosno jednu *delay* promenljivu.

Vreme izvršavanja u softveru dobijamo tako što *offset_soft* promenljivu uvećavamo na svim mestima gde softver pristupa BRAM-u, i to za 10 ciklusa clocka, odnosno 10 puta *delay*.

Kasnije se ova vremena sabiraju u *CPU.cpp* fajlu i njihovo ukupno vreme se ispisuje.

Za slike veličine 225x225 piksela nad kojima smo vršili testiranja smo dobili vreme izvršavanja od 920.860.380ns odnosno 0,92086038s po slici.

Ovom procenom dobijamo da naš sistem u jednoj sekundi može obraditi 1,085940954 odnosno jednu sliku i od nje generisati piramidu gausovih blurova, odnosno niz slika različitih blurova i veličina. Ovo znači da sistem koji će generisati piramidu gausovih blurova može raditi na 1fps (frames per second).